A Proposal for a Tiered Cognitive Architecture for AI Systems

Executive Summary

Current AI systems are built around a remarkably capable computational engine: the large language model (LLM). As these systems have evolved, however, the language model has gradually assumed responsibility for functions extending well beyond probabilistic inference, including conversation state reconstruction, memory management, planning, constraint tracking, tool routing, and execution coordination.

This concentration of responsibility forces fundamentally different classes of computation through the same probabilistic engine. The result is structural inefficiency: repeated reconstruction of conversational state, increasing computational cost, degradation over long interactions, and growing cognitive friction for users who must repeatedly correct incorrect assumptions or redirect the conversation.

This proposal explores an alternative architectural direction.

Rather than treating the language model as the operating system of an AI system, it proposes separating **executive cognition** from **computational inference**.

Executive cognition refers to the functions responsible for maintaining goals, managing working state, selecting relevant information, coordinating specialised computational resources, and integrating results into persistent knowledge. These functions are distinct from language generation or probabilistic reasoning and therefore need not rely on the same computational substrate.

The proposed architecture introduces a persistent **Executive Cognitive Kernel** that maintains structured cognitive state, compiles task-specific execution contexts, and dispatches work to specialised computational engines.

The guiding principle is straightforward:

> **Different classes of cognitive work should be performed by the computational substrate best suited to perform them.**

The Current Architecture and Its Structural Limitations

Modern AI systems already consist of sophisticated infrastructure surrounding a language model, including CPUs, GPUs, retrieval systems, databases, serving engines, orchestration layers, safety systems, and external tools.

Despite this complexity, the language model frequently functions as the system's de facto executive controller. Each interaction reconstructs conversational state, interprets user intent, selects relevant information, performs planning, and generates responses through repeated neural inference.

This proposal argues that many limitations commonly attributed to language models may instead arise from this architectural arrangement.

Reconstructing Working State

Current systems repeatedly ask a probabilistic model to reconstruct its own working state from conversational history.

Working state therefore becomes an inference by-product rather than an explicitly managed system resource.

As conversations grow longer, reconstruction becomes increasingly expensive while simultaneously becoming more susceptible to ambiguity, outdated assumptions, and accumulated errors.

This proposal instead treats working state as an architectural responsibility independent of model inference.

Repeated Context Processing

Conversation histories continually expand while much of their content becomes irrelevant to the current task.

Although modern inference systems employ sophisticated optimisation techniques, repeatedly reconstructing state from growing conversational history remains an architectural overhead independent of any particular transformer implementation.

Context Degradation

Long conversations naturally accumulate abandoned ideas, superseded assumptions, exploratory reasoning, and corrected mistakes.

These elements may continue influencing subsequent inference despite no longer representing the desired cognitive state.

Recovering often requires additional interactions whose primary purpose is correcting previous generations.

Human Productivity Cost

The computational cost of repeated correction is only part of the problem.

Users themselves become the debugging system.

Every unnecessary correction interrupts concentration, pollutes conversational history, increases cognitive load, and reduces productivity.

Architectural Principle

This proposal is not an incremental refinement of existing agentic architectures.

Instead, it relocates executive cognition outside the probabilistic inference engine.

Language generation, logical verification, planning, optimisation, retrieval, symbolic reasoning, memory organisation, and execution coordination represent different computational problems.

They should not necessarily share the same computational mechanism.

Rather than forcing every cognitive function through neural inference, the architecture assigns each class of computation to the substrate most naturally suited to perform it.

Tier One — Executive Cognitive Kernel

The Executive Cognitive Kernel functions as the persistent operating system of the cognitive architecture.

Unlike today's stateless inference model, the Kernel maintains continuous cognitive state independently of any individual model invocation.

Its responsibilities include:

- Maintaining persistent cognitive state
- Tracking goals, projects, and objectives
- Managing structured knowledge
- Selecting relevant information
- Compiling execution contexts
- Dispatching specialised computation
- Integrating returned results
- Coordinating heterogeneous computational resources

Importantly, the Executive Kernel is deterministic.

Its purpose is not to perform every form of reasoning itself.

Its purpose is to coordinate reasoning.

This role is loosely analogous to executive function in biological cognition, which coordinates attention, working memory, and goal management without directly performing every specialised cognitive operation.

Context Compilation

Context compilation is the defining mechanism of the proposed architecture.

The Executive Kernel does not replay conversations.

It compiles context.

Rather than transmitting an entire conversational transcript to a language model, the Kernel constructs a minimal execution payload derived from structured cognitive state.

Each payload contains only the information necessary to complete the current task.

Temporary execution contexts are discarded after completion.

Persistent cognitive state remains independent of model inference.

This separates long-term knowledge from temporary working context while reducing computational overhead and preventing obsolete conversational history from unnecessarily influencing future reasoning.

Persistent Cognitive State

Conversation transcripts are not treated as memory.

Instead, the Executive Kernel maintains structured system state describing the current cognitive environment.

Examples include:

- user preferences
- established facts
- active projects
- current objectives
- unresolved questions
- hypotheses under evaluation
- procedural knowledge
- relationships between concepts

The Executive Kernel owns this state.

Computational engines consume selected portions of it.

Context Compilation as a Research Problem

Context compilation is itself a computational problem.

This proposal defines its architectural role but intentionally does not prescribe a single implementation.

Possible approaches may combine deterministic rules, learned retrieval policies, symbolic reasoning, probabilistic relevance estimation, semantic indexing, graph traversal, or future specialised algorithms.

Determining the optimal compilation strategy remains an open area for empirical research rather than a fixed design assumption.

Tier Two — Specialised Computation Layer

Tier Two consists of heterogeneous computational resources optimised for different classes of computation.

The Executive Kernel determines which computational substrate is appropriate for each task.

Neural Inference Engines

GPUs, NPUs, and future neural accelerators perform:

- language generation
- probabilistic reasoning
- multimodal inference
- pattern recognition
- creative synthesis

These engines become specialised computational resources rather than executive controllers.

Symbolic Reasoning Engines

Deterministic processors implemented through CPUs, ASICs, FPGAs, or future specialised hardware perform operations including:

- rule-based inference
- logical verification
- mathematical reasoning
- constraint satisfaction
- graph traversal
- expert systems

Rather than approximating these operations probabilistically, deterministic systems execute them directly whenever appropriate.

Additional Computational Resources

The architecture naturally extends to additional specialised resources, including:

- search systems
- optimisation engines
- planning algorithms
- simulation engines
- scientific computing platforms
- future specialised accelerators

The architecture is intentionally hardware-agnostic.

It is organised around computational specialisation rather than particular technologies.

Knowledge Infrastructure

Persistent knowledge forms shared cognitive infrastructure owned by the Executive Kernel rather than another computational engine.

Possible implementations include:

- semantic knowledge graphs
- structured key-value state
- relational databases
- vector indices where appropriate
- append-only episodic memory logs

The Executive Kernel determines how these structures are maintained and queried.

Computational engines consume information from them but do not own them.

Execution Dispatch

Not every task requires full orchestration.

Simple conversational requests may be dispatched directly to a neural inference engine.

More complex tasks may involve combinations of:

- retrieval
- symbolic verification
- optimisation
- planning
- simulation
- neural synthesis

The Executive Kernel dynamically assembles the execution pipeline appropriate for each request.

Translation and Trade-offs

Separating executive cognition from inference introduces new engineering challenges.

Translating human language into structured cognitive state is itself imperfect.

Rather than assuming perfect interpretation, the Executive Kernel evaluates confidence before modifying persistent state.

When confidence is insufficient, the system requests clarification before committing changes.

Likewise, front-loaded context compilation introduces computational overhead.

However, this overhead may reduce repeated inference cycles, unnecessary computation, repeated user corrections, and long-term conversational drift.

These represent architectural hypotheses requiring empirical validation.

Architectural Comparison

Dimension	Monolithic LLM	Agentic Systems	Tiered Cognitive Architecture
Executive Control	Neural model	Neural planner	Executive Cognitive Kernel
Working Context	Replay history	Replay + retrieval	Compiled execution context
Persistent State	Conversation transcript	External memory	Structured cognitive state
Primary Reasoning	Neural inference	Neural inference	Specialised heterogeneous computation
Computation	Primarily neural	Primarily neural	Computational specialisation
System State	Ephemeral	Semi-persistent	Persistent and deterministic

---

Research Questions

This proposal is intended as an architectural research agenda rather than a completed implementation.

Key questions include:

- Does compiled execution context reduce inference cost?
- Does structured cognitive state improve long-horizon conversational coherence?
- Can deterministic executive state reduce hallucination and conversational drift?
- What execution-dispatch strategies provide the best trade-off between latency and capability?
- How should context compilation be implemented and evaluated?
- Does heterogeneous computation improve throughput, reliability, or infrastructure utilisation relative to predominantly neural execution?

These questions are empirical and should be evaluated experimentally.

Conclusion

The history of AI has largely focused on constructing increasingly capable models.

This proposal explores a complementary direction: constructing increasingly capable cognitive systems around those models.

Rather than assuming every cognitive function belongs inside a language model, the architecture separates executive cognition from specialised computation.

Language models remain indispensable for probabilistic inference, pattern recognition, and natural language generation.

They simply cease to function as the operating system of the AI.

Whether this architecture ultimately proves superior is an empirical question.

The contribution of this proposal is not a complete implementation but a different organising principle for AI systems:

> **Working state should be an explicitly managed architectural resource rather than an emergent by-product of repeated neural inference.**

If that principle proves correct, future advances in AI may come not only from building larger models, but from building cognitive architectures that coordinate specialised forms of computation through a persistent executive layer capable of compiling structured cognitive state into task-specific execution contexts.